Image Denoising with Bilateral Filters: From a Tuned Classic Filter to a Neural-Guided One

Manas Pandya
NLP & CV Lab, Task 07

October 2026

Live demo: `https://nlp-cv-lab.vercel.app/p/bilateral-denoising`
Code: `https://github.com/manasrpandya/NLP_CV_Lab/tree/main/projects/07-bilateral-denoising`
Model: `https://huggingface.co/manasrpandya/bilateral-denoise-nbf`

1 Aim

The task is to denoise images with bilateral filters. Calling `cv2.bilateralFilter` once with default-looking parameters does remove some noise, but it leaves two questions open: how good can the bilateral filter get if its parameters are chosen properly, and how far is that from methods people actually use for denoising? So the plan was a ladder of methods, each one a row in the results:

(a) the classic bilateral filter with (d, σ_s, σ_r) grid-searched per noise level on held-out images (an oracle, since it is told the true noise level),

(b) the same filter with blind parameters: estimate the noise level from the image and map it to parameters with a fit from (a),

(c) iterated and joint (cross) bilateral filtering,

(d) a "neural-guided bilateral" filter (NBF), where a small CNN predicts σ_s, σ_r and the range guide for every pixel, trained end to end through a differentiable bilateral layer,

(e) reference points that are not bilateral: non-local means, BM3D and a pretrained DnCNN.

Everything is evaluated with PSNR/SSIM on standard test sets at three noise levels plus one real-noise set. The NBF model is exported to ONNX and runs in the browser on the demo page, next to a WebGL version of the classic filter that updates in real time as its parameters are changed.

2 Background

The bilateral filter [1] replaces each pixel p by a normalised weighted average of its neighbours q in a window Ω_p:

$$\hat{x}_p = \frac{\sum_{q\in\Omega_p} w_{pq}\, y_q}{\sum_{q\in\Omega_p} w_{pq}}, \qquad w_{pq} = \exp\Big(-\frac{\|p-q\|^2}{2\sigma_s^2}\Big)\exp\Big(-\frac{(g_p-g_q)^2}{2\sigma_r^2}\Big), \tag{1}$$

where y is the noisy image and g is the image used for the range term. For the plain filter $g = y$; for the joint or cross bilateral filter g is a different (here: smoothed) image. The spatial term makes it a blur, and the range term stops it from averaging across edges. The problem is that with $g = y$ the range term sees the noise too: if σ_r is small compared to the noise, noisy pixels only average

with themselves, and if it is large, the filter turns into a Gaussian blur and edges are lost. There is also one global (σ_s, σ_r) for the whole image, which is a poor fit for images with both flat areas and texture.

This has been attacked in several ways. The adaptive bilateral filter of Gavaskar and Chaudhury [2] lets the range kernel centre and width change per pixel. HDRNet [3] predicts the coefficients of a bilateral grid with a CNN, and bilateral neural networks [4] learn the filter itself in a permutohedral lattice. My NBF model sits closest to the adaptive filter: the output is still Eq. 1, only σ_s, σ_r and g are predicted per pixel by a network. The reference methods are non-local means [5], BM3D [6] and DnCNN [7].

3 Data

Training. The 400 BSD images (180×180, gray) used by DnCNN [7]. The last 32 are held out as a validation set and are used for every parameter search and for model selection, so the test sets are never touched for tuning. DIV2K was not added; BSD400 was enough for a model this small.

Testing. Set12, BSD68 and Kodak24 (converted to gray), with additive Gaussian noise at $\sigma \in \{15, 25, 50\}$ (on the 0–255 scale). Noise is drawn once with a fixed seed per image and σ, and is not clipped, which is the usual convention. As a real-noise test I used 320 of the 1280 SIDD validation crops [10] (every 4th, so all 40 scenes are covered, 256×256, converted to gray), taken from a copy on the Hugging Face Hub. The download script `bden.data` fetches and caches all of it.

4 Method

4.1 (a) Tuned bilateral

Grid search of `cv2.bilateralFilter` over $d \in \{3, \dots, 11\}$, $\sigma_s \in [0.8, 5]$ and $\sigma_r \in [0.8, 6] \cdot \sigma$, at $\sigma = 5, 10, \dots, 60$ on the 32 validation images, maximising mean PSNR. Many window sizes tie once the spatial Gaussian has decayed, so I keep the smallest d within 0.02 dB of the best. The result (table in `results/auto_fit.json`) is quite regular: σ_s grows slowly from 0.8 to 2 px, and the best σ_r is 2.5σ at low noise but climbs to 4σ at $\sigma = 60$, which fits a line $\sigma_r \approx 4.17\sigma - 28.8$. At high noise the best filter is basically a Gaussian blur with a mild edge stop, which already says a lot about the limits of the plain bilateral filter.

4.2 (b) Blind auto parameters

The noise level is estimated from the noisy image with Immerkær's method [8] (mean absolute response of a Laplacian-difference mask) and, for comparison, the wavelet estimate median$(|HH|)/0.6745$ [9]. Both are biased upwards by image texture, so each gets a linear calibration $\hat{\sigma} = a\,\sigma_{raw} + b$ fitted on the validation images. The calibrated estimate is mapped to (d, σ_s, σ_r) by linear interpolation in the table from (a). The same estimator and table are implemented in JavaScript for the demo's auto mode.

4.3 (c) Iterated and joint bilateral

The iterated filter runs the bilateral filter n times with a smaller σ_r; the joint filter uses $g = G_{\sigma_g} * y$, a lightly Gaussian-smoothed copy of the noisy image, for the range term so that it is less fooled by the noise. Both were tuned on the validation set (n, σ_s, σ_r, σ_g) at $\sigma \in \{5, 10, 15, 25, 50\}$ and interpolated in between. Two iterations always won. The joint filter used my PyTorch bilateral layer (below), which matches cv2 to within $5 \cdot 10^{-3}$ (unit test).

4.4 (d) Neural-guided bilateral (NBF)

Bilateral layer. Eq. 1 written in PyTorch with a $k \times k$ disc window ($k = 7$, 37 taps, reflect padding, like cv2). σ_s and σ_r can be scalars or per-pixel maps, and there is an optional guide. The weights are computed in the log domain and normalised with a softmax over the taps, which is the same as Eq. 1 but does not underflow for tiny σ_r. The plan said "unfold-based"; I ended up stacking 37 shifted slices of the padded image instead of `F.unfold`, because that exports to plain `Slice`/`Concat` ONNX ops which every runtime supports (including ONNX Runtime Web on WebGPU), and it costs the same memory.

Network. A small 3-level U-Net (16/32/48 channels, leaky ReLU, strided conv down, bilinear up, 92.7k parameters) takes the noisy image and outputs 5 maps: σ_s and σ_r for two bilateral passes and a guide residual. They go through bounded activations, $\sigma_s = 0.3 + 3.2\,\mathrm{sigmoid}(\cdot)$ px and $\sigma_r = 0.003 + 0.5\,\mathrm{sigmoid}(\cdot - 2)$. Pass 1 filters the noisy image with range weights from the guide $g = y + r$; pass 2 filters the result of pass 1 with itself as guide. So the output is always a convex combination of noisy pixels in a 13×13 neighbourhood: it cannot invent structure that is not there, which a plain CNN can.

Training. Random 80×80 crops with flips and rotations, batch 32, Gaussian noise with $\sigma \sim U(0, 55)$ per crop, so one model is blind to the noise level. L1 loss, AdamW with learning rate $2 \cdot 10^{-3}$, cosine decay over 9000 iterations, gradient clipping at 1. This takes 7.5–15 minutes on the laptop GPU (RTX 2050; the spread is CPU contention from evaluation runs going on at the same time, and the iteration budget is fixed so the runs are comparable). The checkpoint with the best validation PSNR at $\sigma = 25$ is kept. The four models trained (NBF and three ablations) used about 45 minutes of GPU time in total.

Export. The model is exported to ONNX (opset 17, dynamic batch and size) with both the image and the 5 maps as outputs. The file is 462 kB and the max absolute difference to PyTorch on three test images is $7.6 \cdot 10^{-6}$ (checked with the shared `labcore.parity` helper, and in a unit test).

4.5 (e) References

cv2 `fastNlMeansDenoising` with h tuned per σ on validation, the `bm3d` package with the true σ, and the blind gray DnCNN-B weights from the KAIR model zoo [7] (666k parameters, trained on the same BSD400 images with $\sigma \in [0, 55]$, so a fair blind comparison). On SIDD every method that needs a noise level gets the Immerkær estimate.

5 Experiments and Results

		Set12			BSD68			Kodak24			SIDD
	Method	15	25	50	15	25	50	15	25	50	real
-	Noisy input	24.69	20.32	14.76	24.79	20.48	14.91	24.74	20.39	14.78	27.02
a	Bilateral, tuned per sigma	30.35	27.44	24.03	29.71	26.96	23.98	30.38	27.54	24.57	30.47
b	Bilateral, blind auto params	30.29	27.39	24.03	29.60	26.90	23.97	30.31	27.48	24.56	30.47
c	Iterated bilateral	30.45	27.82	24.57	29.81	27.20	24.36	30.62	28.00	24.97	**30.83**
c	Joint bilateral (smoothed guide)	30.88	28.17	24.35	30.00	27.45	24.05	30.81	28.15	24.54	30.75
d	Neural-guided bilateral (ours)	**31.93**	**29.60**	**26.44**	**30.82**	**28.38**	**25.48**	**32.11**	**29.79**	**26.84**	27.27
e	Non-local means (cv2)	30.36	27.82	24.28	29.37	26.68	23.69	30.37	27.65	24.61	32.59
e	BM3D	32.40	30.00	26.73	31.15	28.63	25.71	32.37	29.96	27.02	28.21
e	DnCNN-B (pretrained)	32.68	30.36	27.18	31.62	29.16	26.23	32.78	30.42	27.51	27.48

Table 1: PSNR (dB). Bold is the best bilateral variant per column. (a) and (c) are told the true σ on the synthetic sets; (b), (d) and DnCNN are blind.

Table 1 is the main result. A properly tuned bilateral filter gives about +6.5 dB at $\sigma = 25$ (26.96 dB on BSD68), and the blind version (b) loses about 0.1 dB of that, so the estimator plus lookup table is good enough to replace manual tuning. Iterated and joint filtering add 0.2–0.6 dB; the joint filter is better at low noise and the iterated one at $\sigma = 50$. The NBF model is a clear step up: +1.4 dB over (a) on BSD68 at $\sigma = 25$ and +1.5 dB at $\sigma = 50$. It beats NLM everywhere on the synthetic sets and ends up 0.2–0.5 dB below BM3D (which is told the true noise level). DnCNN-B, with about 7 times the parameters and no bilateral constraint, is still 0.7–0.8 dB better.

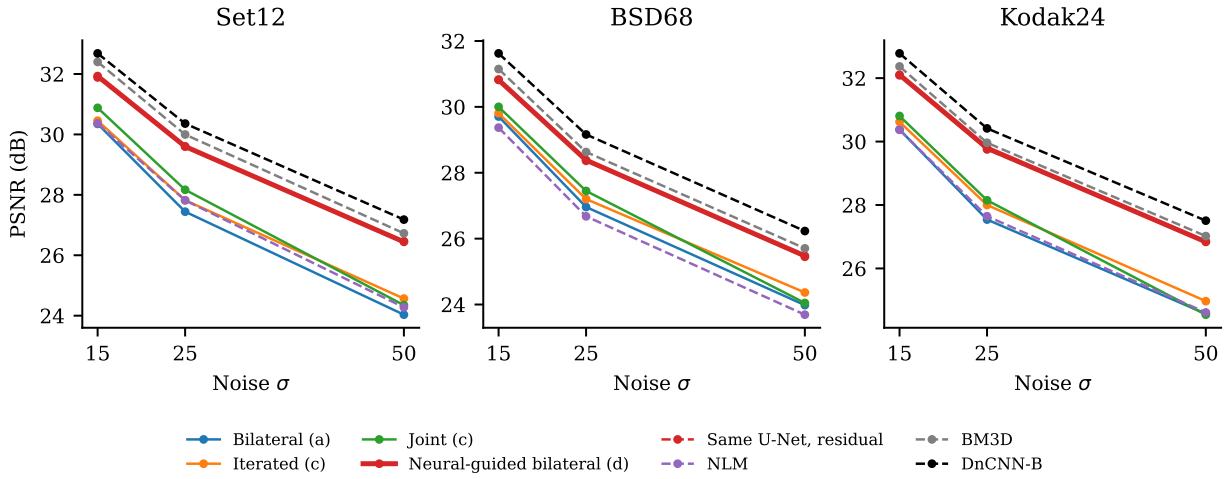


Figure 1: PSNR against the noise level on the three test sets.

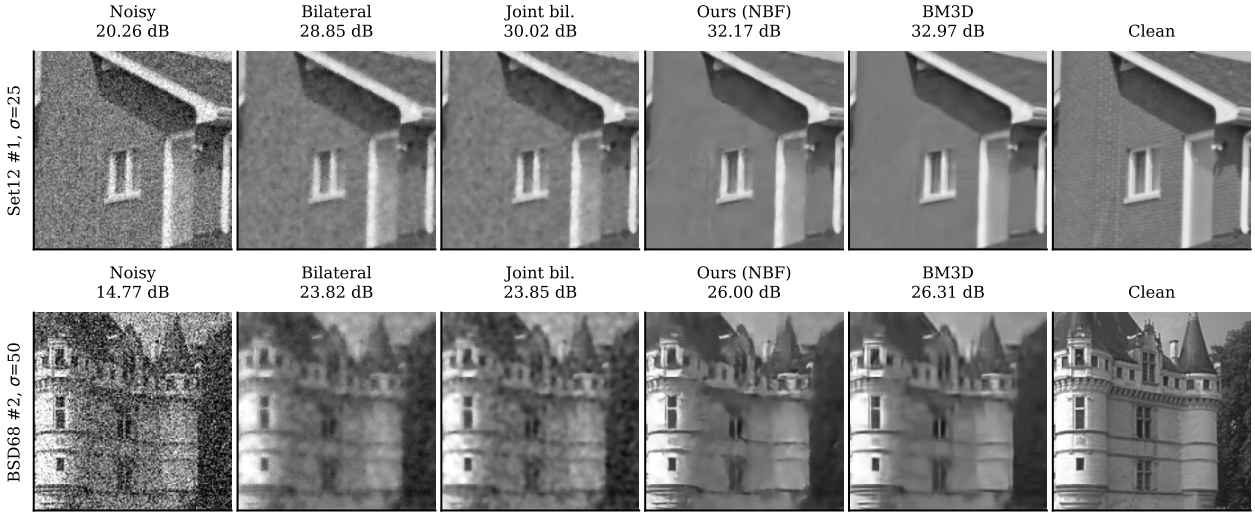


Figure 2: Crops, with the PSNR of the whole image underneath. The plain bilateral filter leaves blotchy noise in flat areas at $\sigma = 25$ and blurs everything at $\sigma = 50$.

SSIM follows the same order (Table 2). The gap between plain bilateral and the rest is larger in SSIM than in PSNR, because the leftover noise in flat areas is exactly what SSIM punishes.

Method	15	25	50	SIDD
Noisy input	0.590	0.416	0.218	0.432
Bilateral, tuned per sigma	0.820	0.711	0.570	0.600
Bilateral, blind auto params	0.815	0.705	0.569	0.600
Iterated bilateral	0.836	0.744	0.596	0.615
Joint bilateral (smoothed guide)	0.836	0.740	0.559	0.612
Neural-guided bilateral (ours)	0.877	0.807	0.682	0.444
Non-local means (cv2)	0.826	0.722	0.579	0.666
BM3D	0.880	0.809	0.691	0.491
DnCNN-B (pretrained)	0.892	0.830	0.719	0.453

Table 2: SSIM on BSD68 and SIDD.

Method noise. Fig. 3 shows $y - \hat{x}$ for one image. Ideally this is pure noise. The plain bilateral filter clearly removes edges (the roof line and the window frames show up), the joint filter a bit less, while the method noise of NBF and BM3D looks like plain noise.

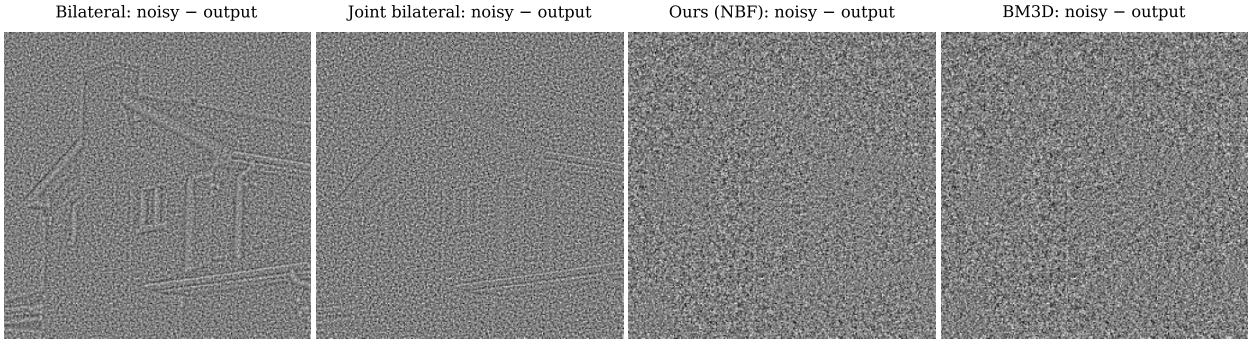


Figure 3: Method noise (noisy minus output) on Set12 house, $\sigma = 25$. Visible structure means detail that the filter removed.

5.1 What the Network Learned

Fig. 4 shows the predicted maps for the first pass. σ_r is low on edges and in texture (the tripod, the coat, the grass) and high in flat regions like the sky, which is the adaptive behaviour I hoped for. σ_s is small along edges and large in flat areas, and the whole map shifts up when the noise goes up. The guide turned out to be interesting: it is not in image units at all, and its values go up to about 4, i.e. the network scales the guide up by about 4×. Since only $(g_p - g_q)/\sigma_r$ matters, this is the same as using a σ_r four times smaller, and it seems the network did this to get around my upper bound on σ_r for pass 1, which most pixels sit close to. That bound was a design mistake, but a harmless one.

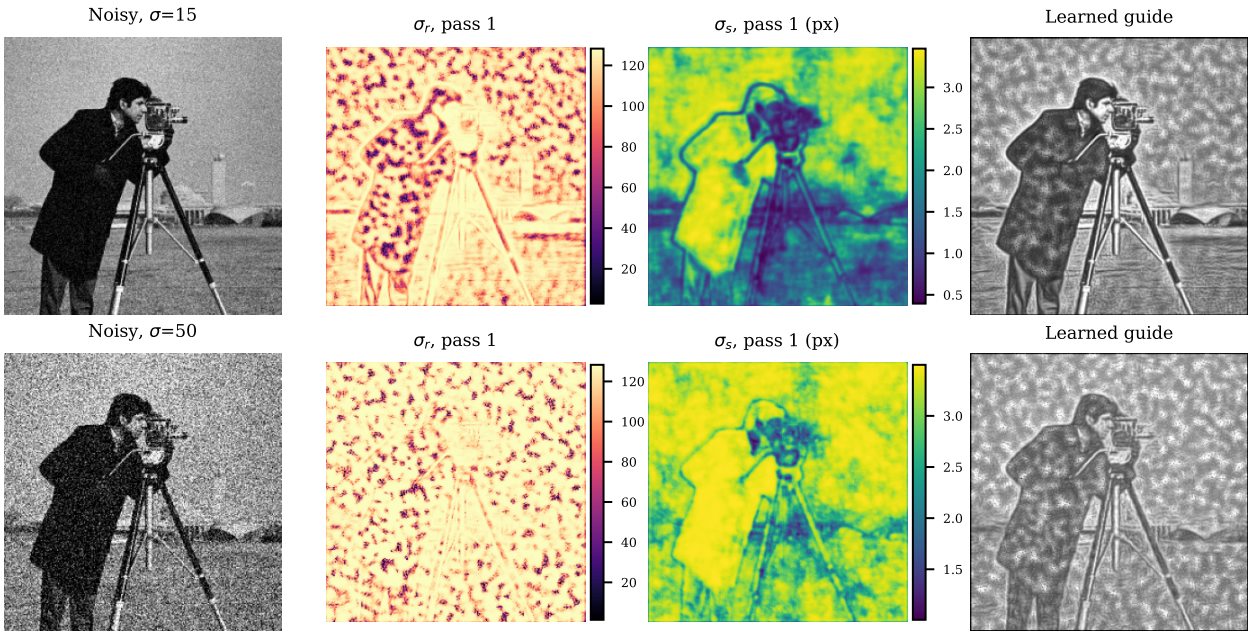


Figure 4: NBF maps for pass 1 on Set12 cameraman at $\sigma = 15$ (top) and 50 (bottom). σ_r on the 0–255 scale, σ_s in pixels.

Fig. 5 averages the maps over 20 BSD68 images at many noise levels. The model never sees σ, but the median pass-1 σ_s goes from 0.3 px at $\sigma = 0$ to 3 px at $\sigma = 60$, and the pass-2 σ_r grows almost linearly with σ, much like the grid search optimum for the classic filter but about 4–5 times smaller. So pass 2 behaves like a classic bilateral filter cleaning up after pass 1, with σ_s pinned at the 3.5 px maximum (with a 7×7 window that is close to a disc box filter). At $\sigma = 0$ the model shrinks pass 1 to almost nothing, so it does not damage clean images.

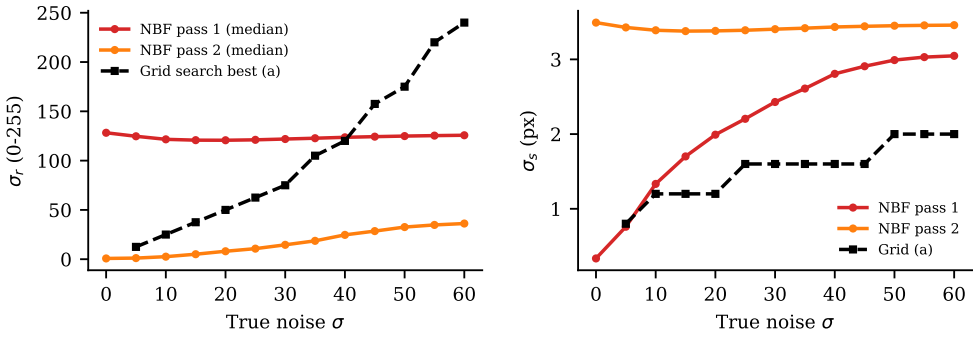


Figure 5: Median predicted σ_r and σ_s against the true noise level, compared with the grid-search optimum of the classic filter (a).

5.2 Ablations

All variants were trained with the same budget (9000 iterations, same seed and data order).

Variant	Params	Val (25)	BSD68 15	25	50
Neural-guided bilateral (ours)	92.7k	29.14	30.82	28.38	25.48
NBF, no guide	92.7k	28.72	30.76	28.28	25.27
NBF, one pass	92.6k	29.02	30.86	28.35	25.19
Same U-Net, plain residual	92.6k	29.17	30.83	28.36	25.45

Table 3: Ablations. "Same U-Net, plain residual" is the same network predicting the noise directly, without any bilateral structure.

The learned guide is the most useful part. Without it (pass 1 uses the noisy image for the range weights, like the plain filter) validation PSNR drops by 0.4 dB; on BSD68 the drop is smaller, 0.06 dB at $\sigma = 15$ growing to 0.21 dB at $\sigma = 50$. The second pass only matters at high noise: with one pass the model is as good or slightly better at $\sigma \leq 25$ but loses 0.29 dB at $\sigma = 50$, where a single 7×7 window is simply too small.

The honest comparison is the last row. The same U-Net trained to predict the noise directly, without any bilateral structure, is just as good: 0.03 dB better on validation and within 0.03 dB on BSD68. So the bilateral constraint does not cost PSNR, but it does not buy any either. What it does give is an output that is always a weighted average of nearby noisy pixels (so it cannot hallucinate texture), parameter maps that can be looked at and explained (Fig. 4), and a model that reduces to almost identity on clean input. The price is runtime: the two bilateral passes roughly double the cost of the U-Net (Table 5). The one place the structure does help is early training (Fig. 6): after 1000 iterations NBF is 0.5 dB ahead of the plain U-Net, which only catches up after about 4000. With a shorter budget or less data the bilateral prior would probably matter more.

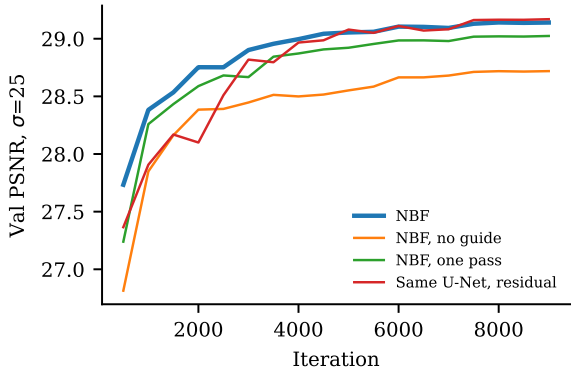


Figure 6: Validation PSNR at $\sigma = 25$ during training.

5.3 Noise Estimation

Estimator	15	25	50
Immerkaer	0.81 (-0.34)	0.67 (-0.43)	0.29 (-0.08)
Wavelet	0.95 (-0.29)	0.84 (-0.41)	0.54 (-0.24)

Table 4: Mean absolute error (and bias) of the calibrated noise estimators on all synthetic test images.

After calibration both estimators are within about 1 grey level of the true σ on Gaussian noise (Table 4, Fig. 7). Immerkær is a bit better and much cheaper, so it is the one used for (b) and in the browser. On SIDD it is a different story, see below.

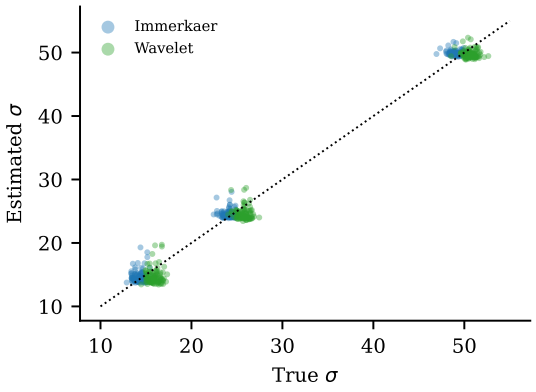


Figure 7: Calibrated estimate against the true σ on all synthetic test images.

5.4 Runtime

Method	CPU (ms)	GPU (ms)	Params
Bilateral, tuned per sigma	12	-	-
Iterated bilateral	14	-	-
Joint bilateral (smoothed guide)	71	-	-
Non-local means (cv2)	162	-	-
BM3D	5040	-	-
Neural-guided bilateral (ours)	291*	27.5	93k
Same U-Net, plain residual	154*	13.1	93k
DnCNN-B (pretrained)	2506*	134.4	666k
NBF, ONNX Runtime	501*	-	-

Table 5: Time for one 512×512 gray image, median of 5 runs. CPU is one thread, * means 4 threads. GPU is the RTX 2050. The machine was not idle (another job was running), so treat these as rough.

The plain bilateral filter is by far the cheapest (Table 5), and in the browser the WebGL shader re-filters a 768×512 image fast enough to follow a slider in real time. NBF is about 5 times faster than DnCNN on the GPU and 9 times on CPU. About half of its GPU time is the two bilateral passes (the plain U-Net alone takes 13 ms); the 37 shifted copies per pass are memory bound rather than compute bound.

6 Discussion and Limitations

Where bilateral filtering loses. Even tuned perfectly, the plain bilateral filter is 1.7 dB behind BM3D at $\sigma = 25$ and 2.2 dB behind DnCNN. It only looks at a small window and its range test is done on noisy values, so at high noise it has to choose between leaving noise and blurring, and the grid search says blurring wins (the best σ_r is 4σ). The iterated and joint versions help a little, but they do not change the picture.

Real noise. On SIDD the ranking flips. The Gaussian-trained networks barely change the image (NBF +0.25 dB, DnCNN +0.5 dB), while the classical filters driven by the Immerkær estimate gain 3.5 dB and NLM 5.6 dB. Real smartphone noise after the camera pipeline is spatially correlated and signal dependent. The residual between noisy and clean SIDD crops has a median standard deviation of 9.9 grey levels, but Immerkær estimates 4.2, because most of the noise energy is at lower frequencies than the Laplacian mask looks at. The networks were trained on white noise, see little high-frequency noise, and treat the rest as texture. The classical filters still smooth because

they are told some noise is present. Training NBF on realistic noise (e.g. SIDD medium training pairs or a simple correlated-noise model) would be the obvious next step and was out of the time budget here.

Color. Everything was evaluated on gray images. In the demo the filters run per channel on RGB, which works but leaves some chroma noise; a joint color range kernel would be better.

Browser. The browser runs NBF on 256×256 tiles with 24 px of context on each side to keep memory low on phones. On the WASM fallback a 768×512 RGB image took about 5 s on my (busy) laptop. WebGPU is faster where it works, and the worker falls back to WASM if a WebGPU warm-up run fails or hangs (which happened with a software adapter in headless Chrome).

7 How to Run

```
cd projects/07-bilateral-denoising
uv sync --extra ref
uv run python -m bden.data          # data + dncnn weights into data/
uv run python -m bden.search        # (a)/(b) grid search + estimator calibration
uv run python -m bden.tune          # (c) and nlm params
uv run python -m bden.train --mdl nbf
uv run python -m bden.evaluate --methods noisy bf_oracle bf_auto bf_iter \
    bf_joint nlm bm3d nbf dncnn
uv run python -m bden.export && uv run python -m bden.bench
uv run python -m bden.figs && uv run python -m bden.tables
uv run pytest -q
```

The demo page source is in `web/app/p/bilateral-denoising` and `web/lib/bilateral-denoising` of the same repository.

References

[1] C. Tomasi and R. Manduchi. Bilateral filtering for gray and color images. ICCV 1998.

[2] R. G. Gavaskar and K. N. Chaudhury. Fast adaptive bilateral filtering. IEEE TIP 28(2), 2019.

[3] M. Gharbi, J. Chen, J. T. Barron, S. W. Hasinoff and F. Durand. Deep bilateral learning for real-time image enhancement. SIGGRAPH 2017.

[4] V. Jampani, M. Kiefel and P. V. Gehler. Learning sparse high dimensional filters: image filtering, dense CRFs and bilateral neural networks. CVPR 2016.

[5] A. Buades, B. Coll and J.-M. Morel. A non-local algorithm for image denoising. CVPR 2005.

[6] K. Dabov, A. Foi, V. Katkovnik and K. Egiazarian. Image denoising by sparse 3-D transform-domain collaborative filtering. IEEE TIP 16(8), 2007.

[7] K. Zhang, W. Zuo, Y. Chen, D. Meng and L. Zhang. Beyond a Gaussian denoiser: residual learning of deep CNN for image denoising. IEEE TIP 26(7), 2017.

[8] J. Immerkær. Fast noise variance estimation. CVIU 64(2), 1996.

[9] D. L. Donoho and I. M. Johnstone. Ideal spatial adaptation by wavelet shrinkage. Biometrika 81(3), 1994.

[10] A. Abdelhamed, S. Lin and M. S. Brown. A high-quality denoising dataset for smartphone cameras. CVPR 2018.